

ECE 2011

Introduction to Computer Engineering
Spring 2026

Lecture 21

04-21-2026

Linghao Song

Yale ECE

Problem Set 2

- Problem Set 2:
 - April 23 Thursday
 - Select a slot of 75 min at your convenience
 - ALU, MIPS instructions, single-cycle processor, pipeline, memory (Lec 12 - 20)
 - Prepare your cheat sheet: up to 4 pages, scan or take clear photos of your cheat sheet and solutions
 - Submission no later than 11:59pm ET April 23 , submit to Canvas, so you should start working on your pset no later than 10:44pm.
 - No discussion, internet access, textbook, or slides.
 - You may use a non-programmable calculator.



100	$A \& \sim B$
101	$A \mid \sim B$
110	$A - B$



Assembly Language

- **Instructions:** commands in a computer's language
 - Assembly language: human-readable format of instructions
 - Machine language: computer-readable format (1's and 0's)
- **MIPS:**
 - MIPS has 32 32-bit registers
 - Registers are faster than memory
 - MIPS called "32-bit architecture" because it operates on 32-bit data
- Operand location: physical location in computer
 - Registers
 - Memory
 - Constants (also called *immediates*)

X86 (laptop, servers) [CISC]
ARM (mobile devices) [RISC]
RISC-V, MIPS ⇒ academia/teaching [RISC]
Embedded systems

MIPS Register Set

Name	Register Number	Usage
\$0	0	the constant value 0
\$at	1	assembler temporary
\$v0-\$v1	2-3	Function return values
\$a0-\$a3	4-7	Function arguments
\$t0-\$t7	8-15	temporaries
\$s0-\$s7	16-23	saved variables
\$t8-\$t9	24-25	more temporaries
\$k0-\$k1	26-27	OS temporaries
\$gp	28	global pointer
\$sp	29	stack pointer
\$fp	30	frame pointer
\$ra	31	Function return address

12:26 4月22日周三

– Constants (also called immediates)

© Digital Design and Computer Architecture

MIPS Register Set

Name	Register Number	Usage
\$0	0	the constant value 0
\$at	1	assembler temporary
\$v0-\$v1	2-3	Function return values
\$a0-\$a3	4-7	Function arguments
\$t0-\$t7	8-15	temporaries
\$s0-\$s7	16-23	saved variables
\$t8-\$t9	24-25	more temporaries
\$k0-\$k1	26-27	OS temporaries
\$gp	28	global pointer
\$sp	29	stack pointer
\$fp	30	frame pointer
\$ra	31	Function return address

© Digital Design and Computer Architecture, 2nd Edition, 2012

ELSEVIER

these all
live in the
register
file.

an example to understand how these registers are used:

$$f(x,y) = (x+y) \times (x-y)$$

main:

li \$a0, 5

li \$a1, 3

jal compute

move \$t0, \$v0

done.

compute:

addi \$sp, \$sp, -8

sw \$ra, 4(\$sp)

sw \$s0, 0(\$sp)

not necessary (move \$s0, \$a0)

add \$t0, \$a0, \$a1

sub \$t1, \$a0, \$a1

mul \$v0, \$t0, \$t1

return value capt

function arguments.

jump to address & compute
save the return address in \$ra.

store result in \$t0

this is in register file → points to an address in data memory

make space on stack.

save x in \$s0

Address

0x7ffffc

0x7ffff8

0x7ffff4

(old top)

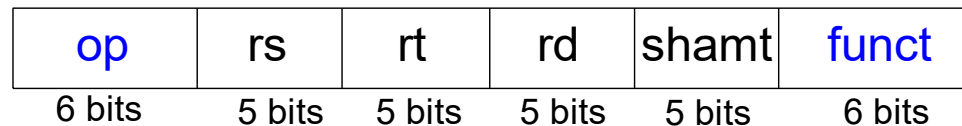
address gets smaller → \$sp

addi \$sp, \$sp, 8 → clean up stack
jr \$ra.

R-Type

- *Register-type*
- 3 register operands:
 - rs, rt: source registers
 - rd: destination register
- Other fields:
 - op: the *operation code* or *opcode* (0 for R-type instructions)
 - funct: the *function*
with opcode, tells computer what operation to perform
 - shamt: the *shift amount* for shift instructions, otherwise it's 0

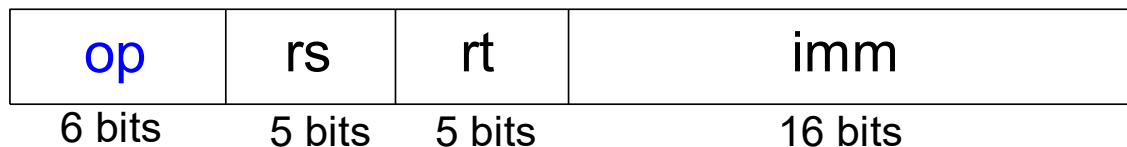
R-Type



I-Type

- *Immediate-type*
- 3 operands:
 - `rs, rt`: register operands
 - `imm`: 16-bit two's complement immediate
- Other fields:
 - `op`: the opcode
 - Simplicity favors regularity: all instructions have opcode
 - Operation is completely determined by opcode

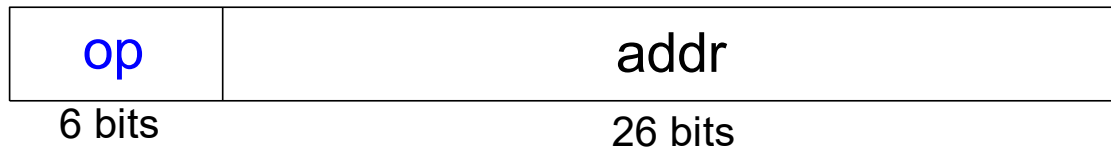
I-Type



Machine Language: J-Type

- *Jump-type*
- 26-bit address operand (`addr`)
- Used for jump instructions (`j`)

J-Type



Review: Instruction Formats

R-Type

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

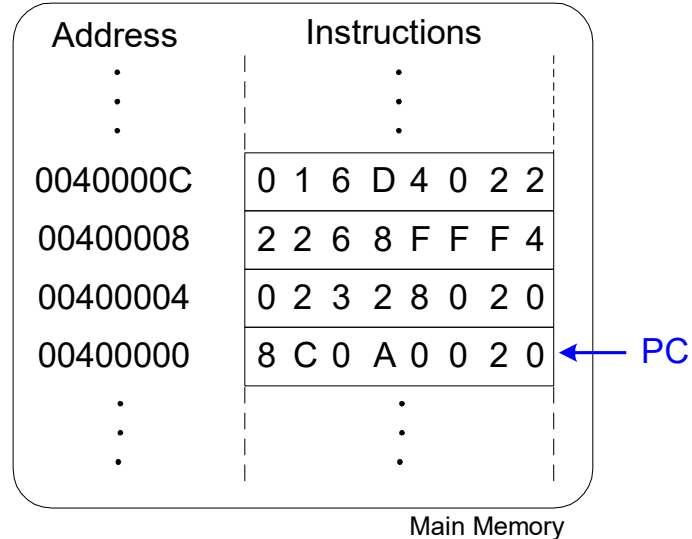
J-Type

op	addr
6 bits	26 bits

The Stored Program

Assembly Code	Machine Code
lw \$t2, 32(\$0)	0x8C0A0020
add \$s0, \$s1, \$s2	0x02328020
addi \$t0, \$s3, -12	0x2268FFF4
sub \$t0, \$t3, \$t5	0x016D4022

Stored Program



Program Counter (PC): keeps track of current instruction

Branching

- Execute instructions out of sequence
- Types of branches:
 - **Conditional**
 - branch if equal (beq)
 - branch if not equal (bne)
 - **Unconditional**
 - jump (j)

Programming

- High-level languages:
 - e.g., C, Java, Python
 - Written at higher level of abstraction
- Common high-level software constructs:
 - if/else statements
 - for loops
 - while loops
 - arrays
 - function calls

Conditional Branching (beq)

MIPS assembly

```
addi $s0, $0, 4      # $s0 = 0 + 4 = 4
addi $s1, $0, 1      # $s1 = 0 + 1 = 1
sll  $s1, $s1, 2      # $s1 = 1 << 2 = 4
beq  $s0, $s1, target # branch is taken
addi $s1, $s1, 1      # not executed
sub  $s1, $s1, $s0     # not executed

target:               # label
add  $s1, $s1, $s0     # $s1 = 4 + 4 = 8
```

Labels indicate instruction location. They can't be reserved words and must be followed by colon (:)

The Branch Not Taken (bne)

MIPS assembly

addi	\$s0, \$0, 4	# \$s0 = 0 + 4 = 4
addi	\$s1, \$0, 1	# \$s1 = 0 + 1 = 1
sll	\$s1, \$s1, 2	# \$s1 = 1 << 2 = 4
bne	\$s0, \$s1, target	# branch not taken
addi	\$s1, \$s1, 1	# \$s1 = 4 + 1 = 5
sub	\$s1, \$s1, \$s0	# \$s1 = 5 - 4 = 1
target:		
add	\$s1, \$s1, \$s0	# \$s1 = 1 + 4 = 5

Unconditional Branching (j)

MIPS assembly

```
addi $s0, $0, 4           # $s0 = 4
addi $s1, $0, 1           # $s1 = 1
j      target             # jump to target
sra    $s1, $s1, 2         # not executed
addi    $s1, $s1, 1        # not executed
sub     $s1, $s1, $s0      # not executed

target:
add     $s1, $s1, $s0      # $s1 = 1 + 4 = 5
```


High-Level Code Constructs

- `if` statements
- `if/else` statements
- `while` loops
- `for` loops

If/Else Statement

C Code

```
if (i == j)
    f = g + h;
else
    f = f - i;
```

MIPS assembly code

If/Else Statement

C Code

```
if (i == j)
    f = g + h;
else
    f = f - i;
```

MIPS assembly code

```
# $s0 = f, $s1 = g, $s2 = h
# $s3 = i, $s4 = j
    bne $s3, $s4, L1
    add $s0, $s1, $s2
    j   done
L1:   sub $s0, $s0, $s3
done:
```

don't forget this!

For Loops

```
for (initialization; condition; loop operation)  
    statement
```

- **initialization:** executes before the loop begins
- **condition:** is tested at the beginning of each iteration
- **loop operation:** executes at the end of each iteration
- **statement:** executes each time the condition is met

For Loops Less Than Comparison

C Code

```
// add the powers of 2 from 1
// to 100
int sum = 0;
int i;

for (i=1; i < 101; i = i*2) {
    sum = sum + i;
}
```

MIPS assembly code

```
# $s0 = i, $s1 = sum
        addi $s1, $0, 0
        addi $s0, $0, 1
        addi $t0, $0, 101
loop:   slt  $t1, $s0, $t0
        beq  $t1, $0, done
        add  $s1, $s1, $s0
        sll  $s0, $s0, 1
        j    loop
done:
```

\$t1 = 1 if $i < 101$

Arrays

- Access large amounts of similar data
- **Index:** access each element
- **Size:** number of elements

Arrays

- 5-element array
- **Base address** = 0x12348000 (address of first element, `array[0]`)
- First step in accessing an array: load base address into a register

0x12340010	array[4]
0x1234800C	array[3]
0x12348008	array[2]
0x12348004	array[1]
0x12348000	array[0]

Accessing Arrays

// C Code

```
int array[5];  
array[0] = array[0] * 2;  
array[1] = array[1] * 2;
```

MIPS assembly code

\$s0 = array base address

```
lui    $s0, 0x1234           # 0x1234 in upper half of $s0  
ori    $s0, $s0, 0x8000      # 0x8000 in lower half of $s0
```

```
lw     $t1, 0($s0)           # $t1 = array[0]  
sll    $t1, $t1, 1           # $t1 = $t1 * 2  
sw     $t1, 0($s0)           # array[0] = $t1
```

```
lw     $t1, 4($s0)           # $t1 = array[1]  
sll    $t1, $t1, 1           # $t1 = $t1 * 2  
sw     $t1, 4($s0)           # array[1] = $t1
```


Arrays using For Loops

// C Code

```
int array[1000];  
int i;  
  
for (i=0; i < 1000; i = i + 1)  
    array[i] = array[i] * 8;
```

MIPS assembly code

```
# $s0 = array base address, $s1 = i
```

Arrays Using For Loops

MIPS assembly code

\$s0 = array base address, \$s1 = i

initialization code

lui \$s0, 0x23B8 # \$s0 = 0x23B80000

ori \$s0, \$s0, 0xF000 # \$s0 = 0x23B8F000

addi \$s1, \$0, 0 # i = 0

addi \$t2, \$0, 1000 # \$t2 = 1000

loop:

slt \$t0, \$s1, \$t2 # i < 1000?

beq \$t0, \$0, done # if not then done

sll \$t0, \$s1, 2 # \$t0 = i * 4 (byte offset)

add \$t0, \$t0, \$s0 # address of array[i]

lw \$t1, 0(\$t0) # \$t1 = array[i]

sll \$t1, \$t1, 3 # \$t1 = array[i] * 8

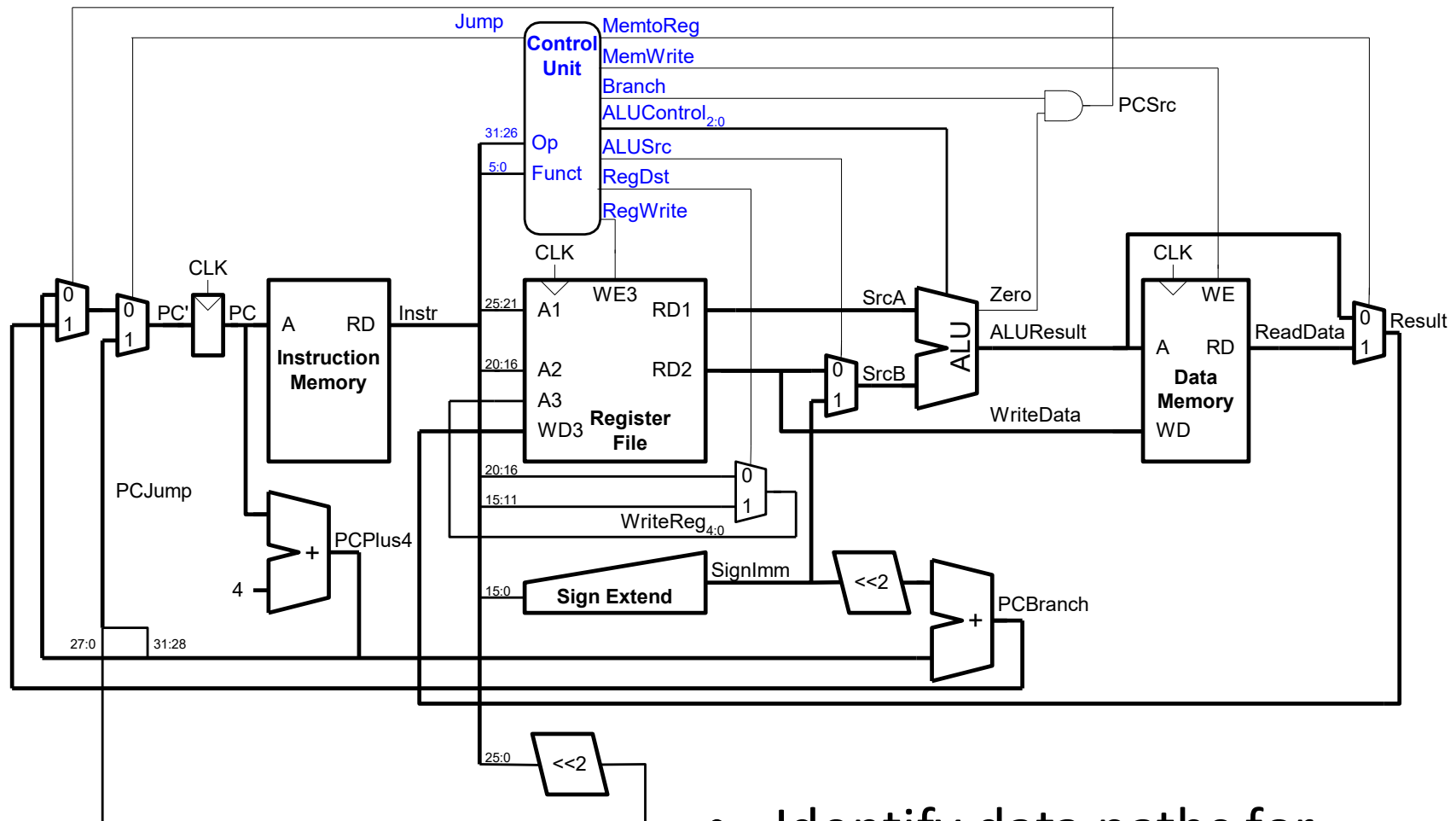
sw \$t1, 0(\$t0) # array[i] = array[i] * 8

addi \$s1, \$s1, 1 # i = i + 1

j loop # repeat

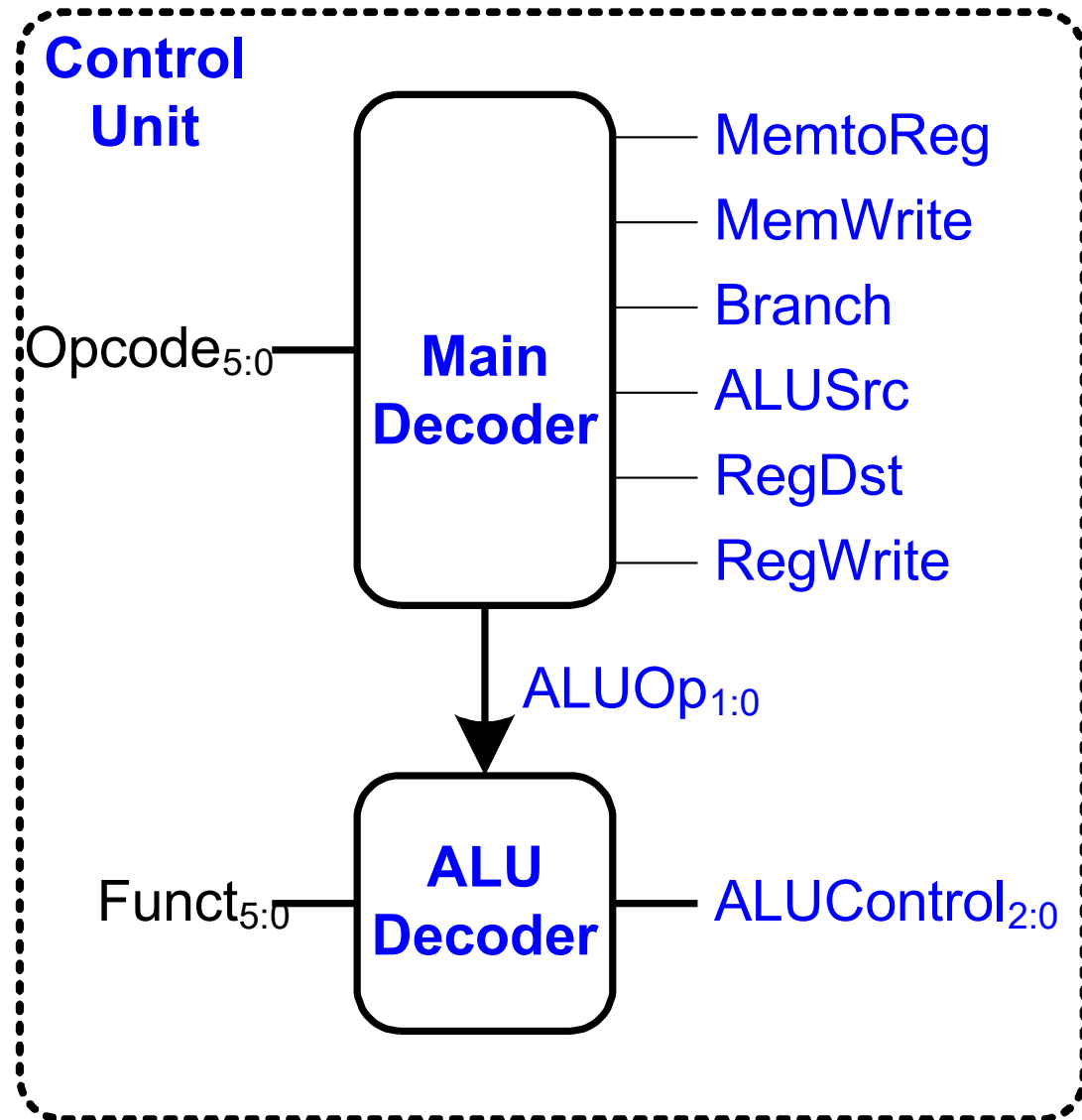
done:

Single-Cycle Processor



- Identify data paths for R, I, and branch instructions

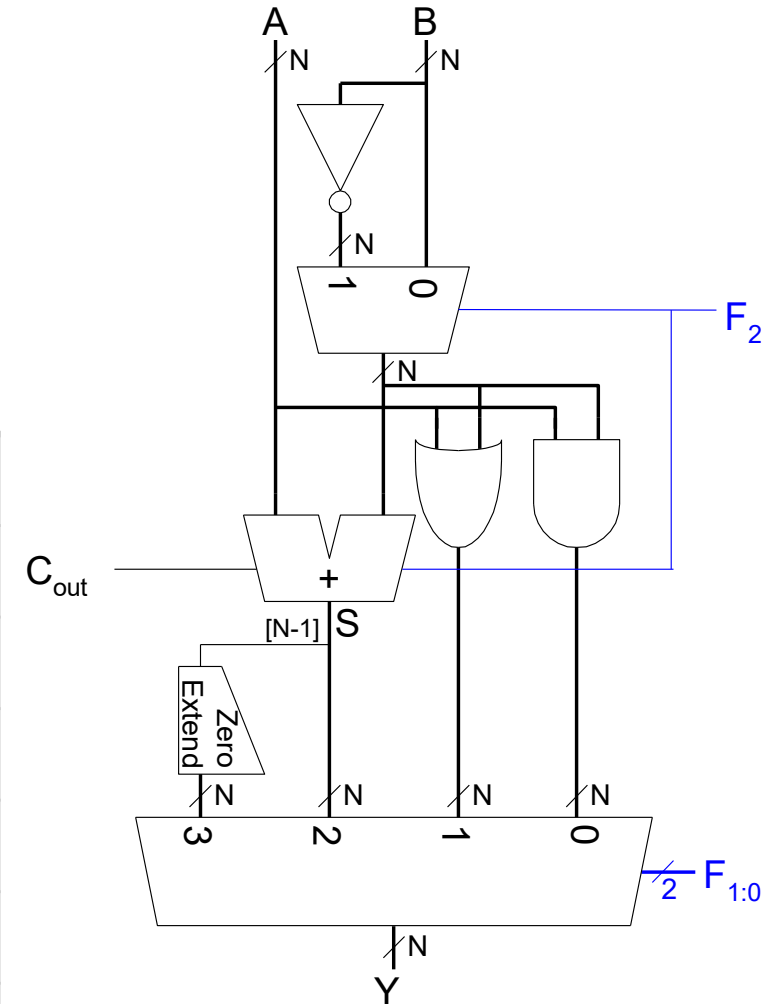
Single-Cycle Control



Control Unit: ALU Decoder

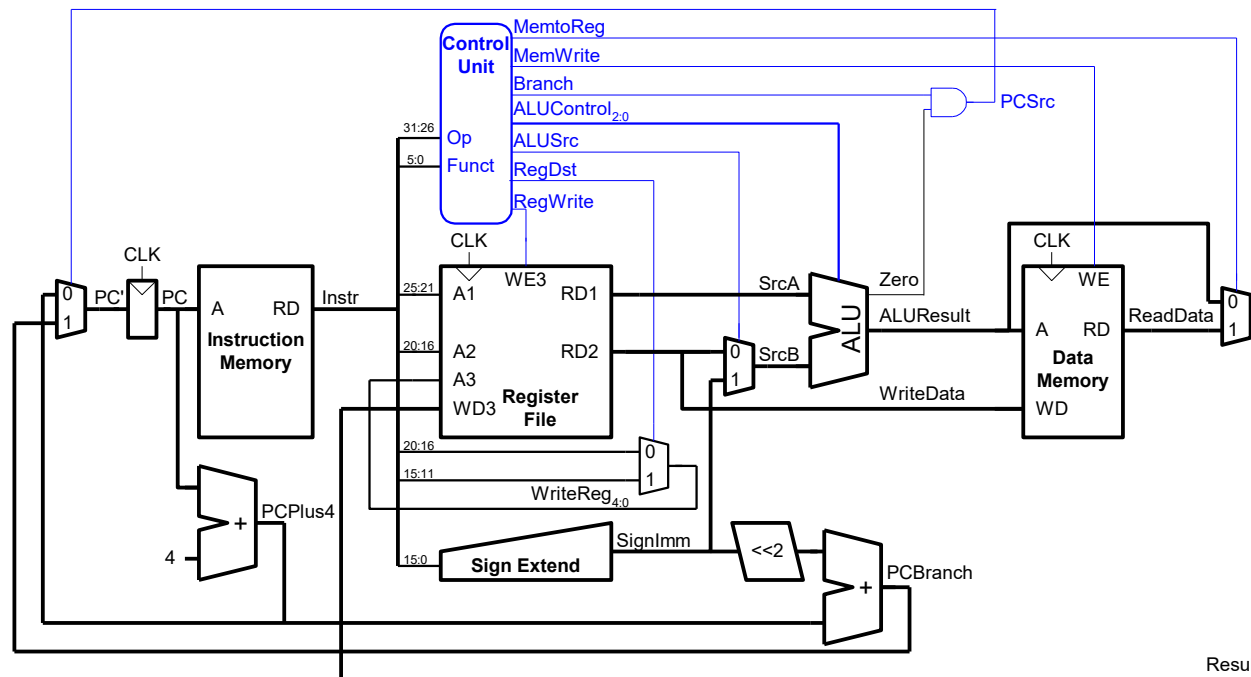
ALUOp _{1:0}	Meaning
00	Add
01	Subtract
10	Look at Funct
11	Not Used

ALUOp _{1:0}	Funct	ALUControl _{2:0}
00	X	010 (Add)
X1	X	110 (Subtract)
1X	100000 (add)	010 (Add)
1X	100010 (sub)	110 (Subtract)
1X	100100 (and)	000 (And)
1X	100101 (or)	001 (Or)
1X	101010 (slt)	111 (SLT)

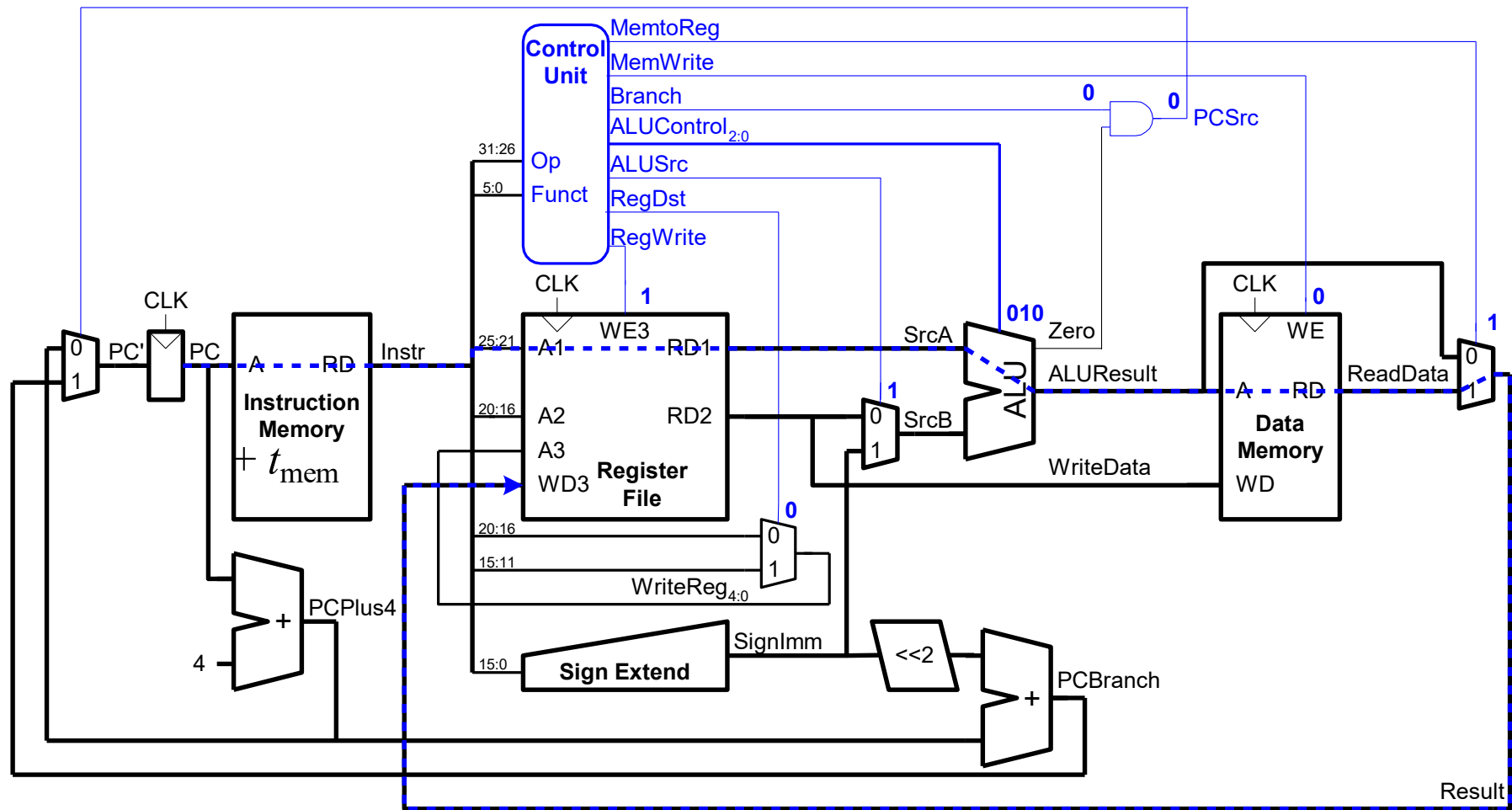


Control Unit Main Decoder

Instruction	Op _{5:0}	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	ALUOp _{1:0}
R-type	000000	1	1	0	0	0	0	10
lw	100011	1	0	1	0	0	0	00
sw	101011	0	X	1	0	1	X	00
beq	000100	0	X	0	1	0	X	01



Single-Cycle Performance



T_C limited by critical path (1w)

Single-Cycle Performance

- Single-cycle critical path:

$$T_c = t_{pcq_PC} + t_{mem} + \max(t_{RFread}, t_{sext} + t_{mux}) + t_{ALU} + t_{mem} + t_{mux} + t_{RFsetup}$$

- Typically, limiting paths are:

- memory, ALU, register file

- $T_c = t_{pcq_PC} + t_{mem} + t_{RFread} + t_{ALU} + t_{mem} + t_{mux} + t_{RFsetup}$

$$= t_{pcq_PC} + 2t_{mem} + t_{RFread} + t_{mux} + t_{ALU} + t_{RFsetup}$$

Single-Cycle Performance Example

Element	Parameter	Delay (ps)
Register clock-to-Q	t_{pcq_PC}	30
Register setup	t_{setup}	20
Multiplexer	t_{mux}	25
ALU	t_{ALU}	200
Memory read	t_{mem}	250
Register file read	t_{RFread}	150
Register file setup	$t_{RFsetup}$	20

$$T_c = ?$$

$$= t_{pcq_PC} + 2t_{mem} + t_{RFread} + t_{mux} + t_{ALU} + t_{RFsetup}$$

$$= [30 + 2(250) + 150 + 25 + 200 + 20] \text{ ps}$$

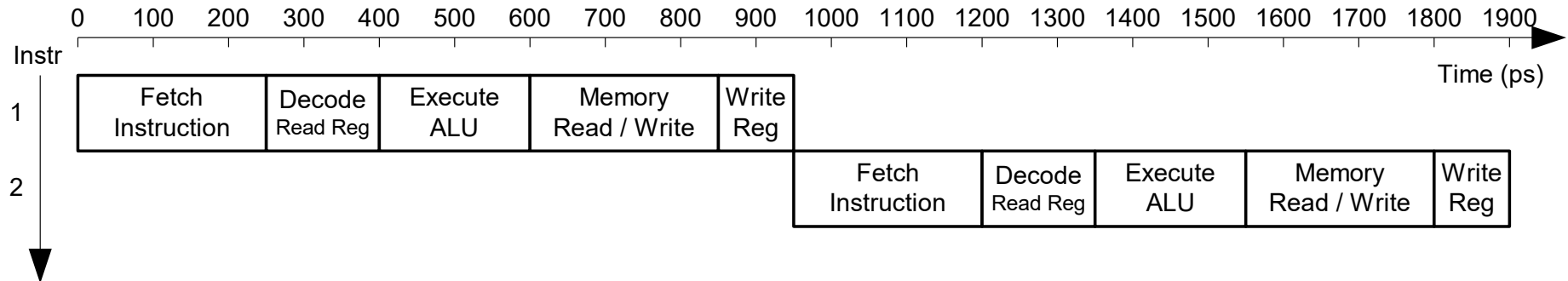
$$= 925 \text{ ps}$$

Pipelined MIPS Processor

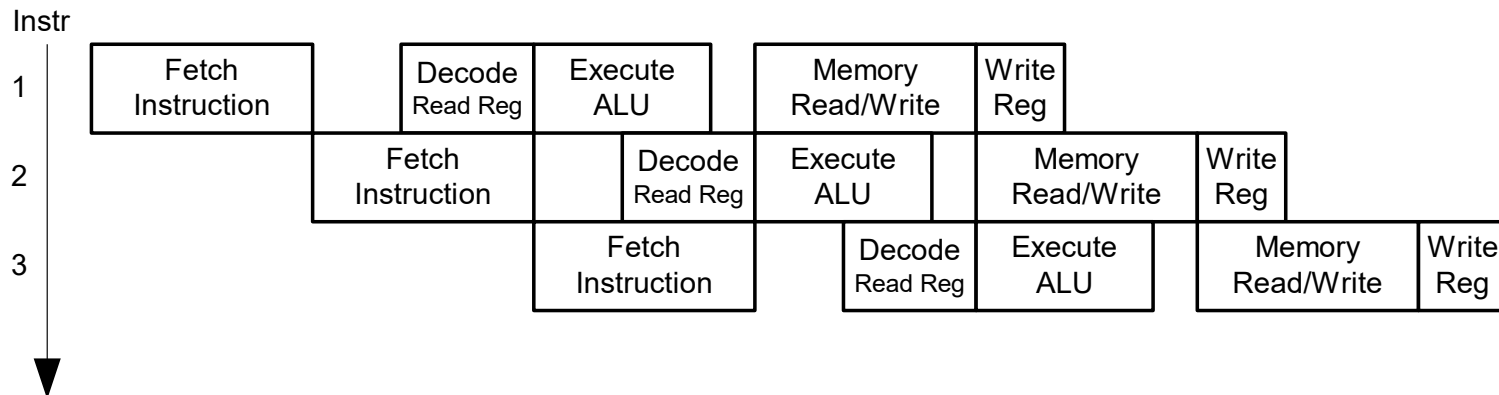
- Temporal parallelism
- Divide single-cycle processor into 5 stages:
 - Fetch
 - Decode
 - Execute
 - Memory
 - Writeback
- Add pipeline registers between stages

Single-Cycle vs. Pipelined

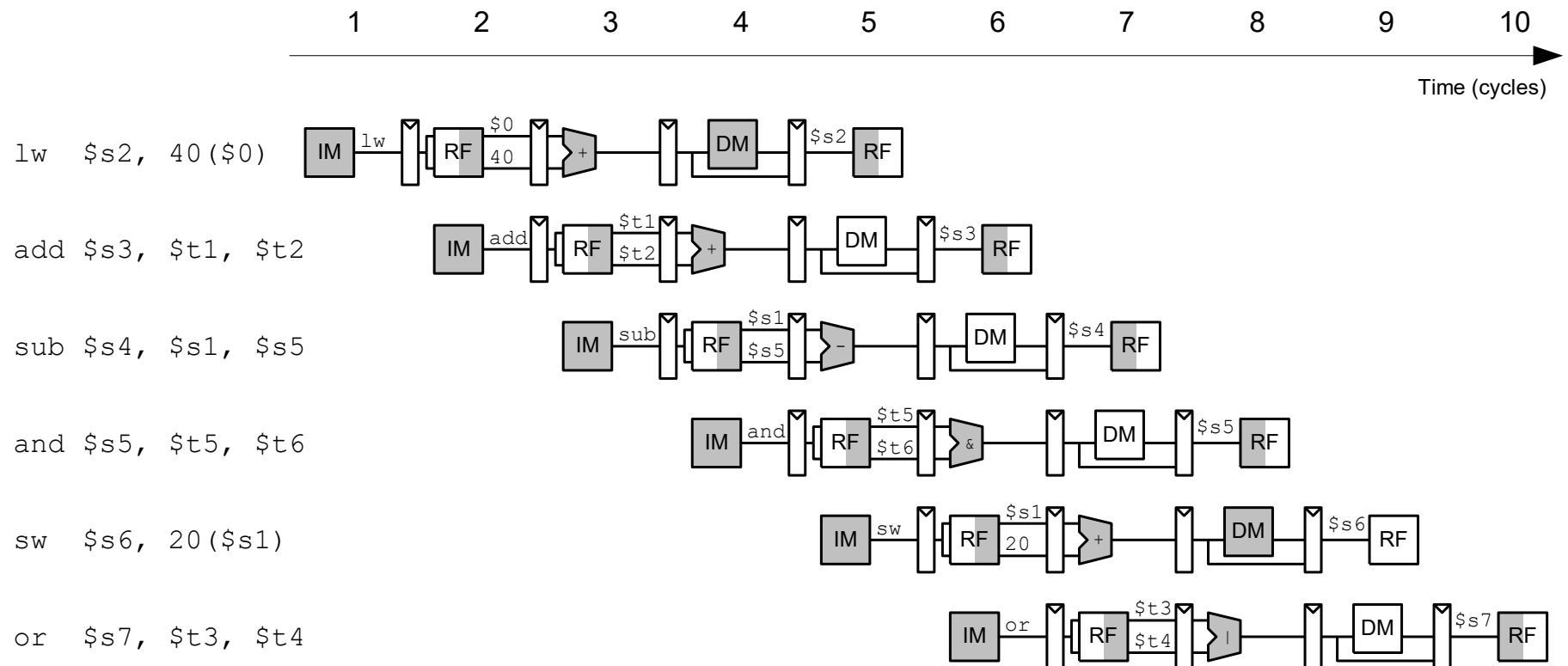
Single-Cycle



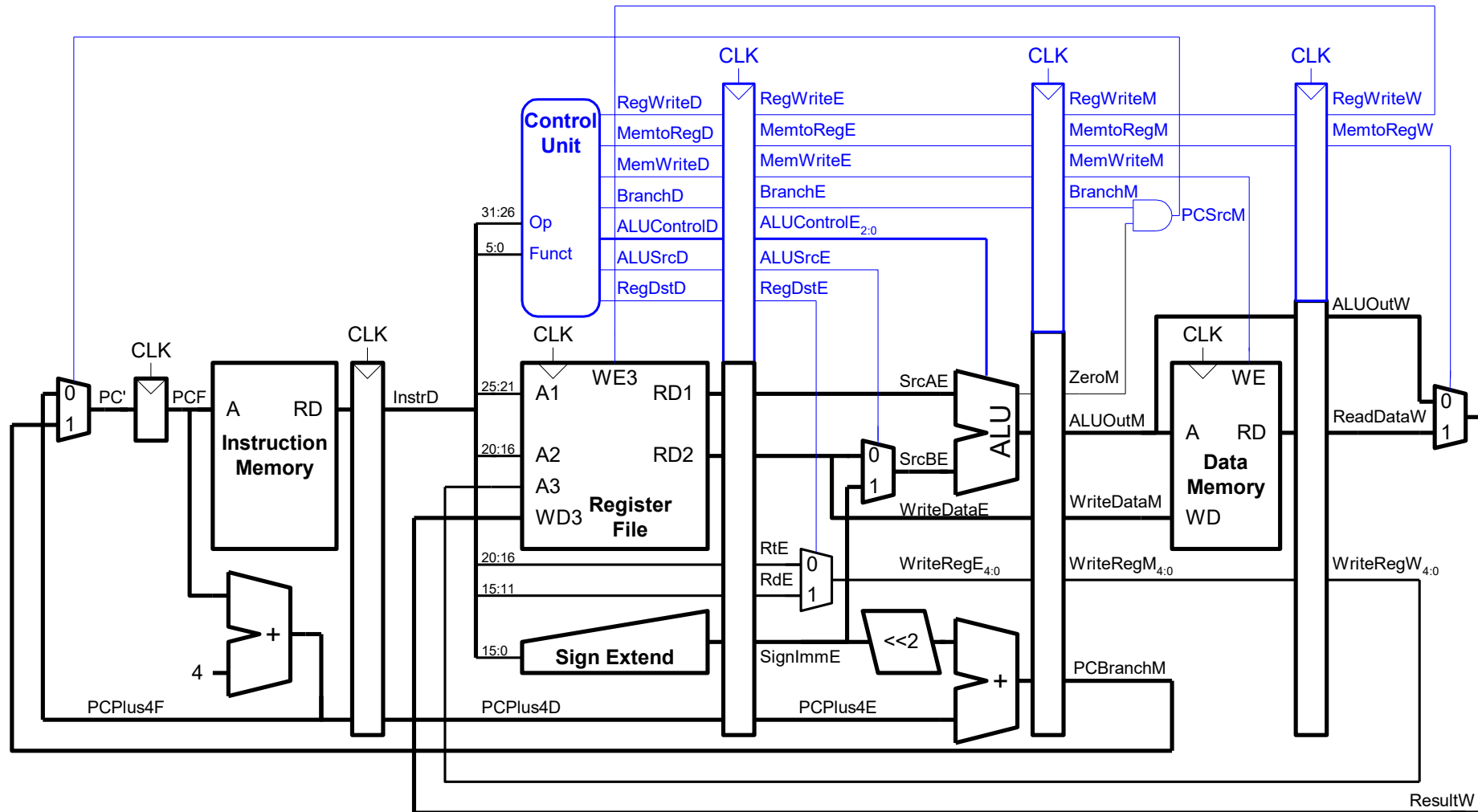
Pipelined



Pipelined Processor Abstraction



Pipelined Processor Control

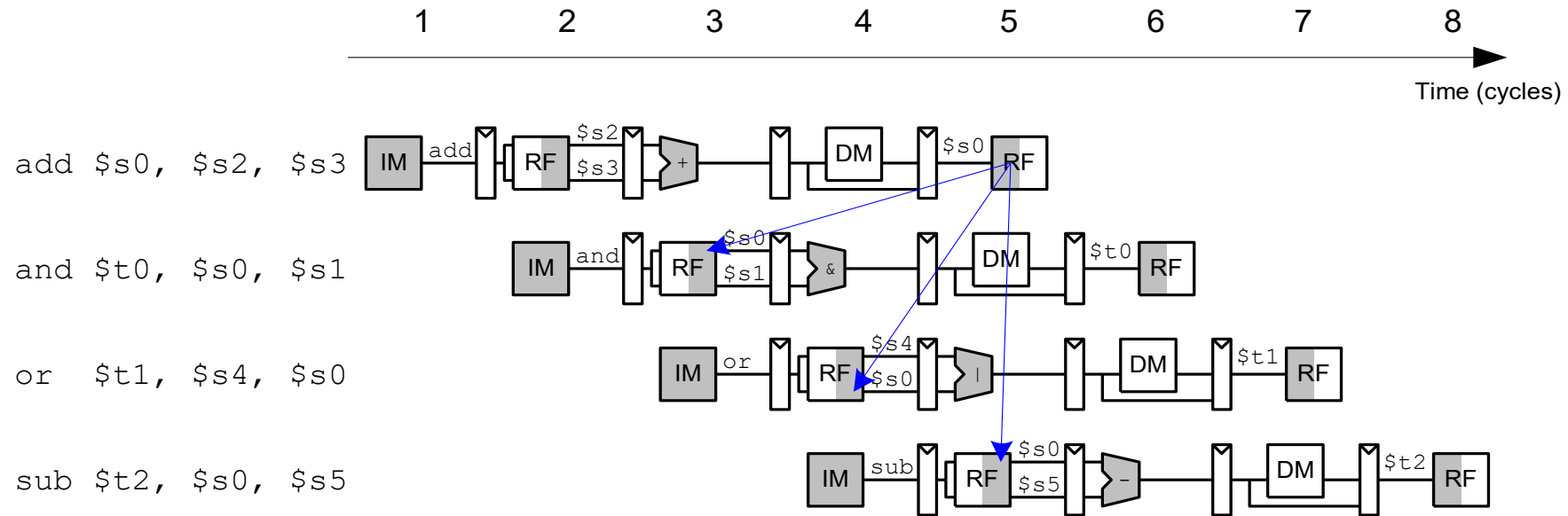


- Same control unit as single-cycle processor
- Control delayed to proper pipeline stage

Pipeline Hazards

- When an instruction depends on result from instruction that hasn't completed
- Types:
 - **Data hazard:** register value not yet written back to register file
 - **Control hazard:** next instruction not decided yet (caused by branches)

Data Hazard



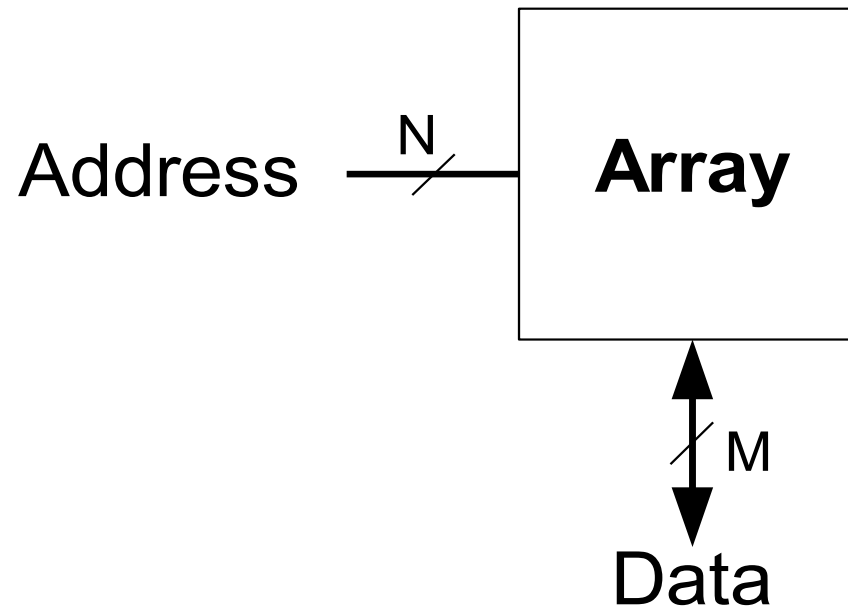
Pipelined Performance

- Pipelined processor critical path:

$$T_c = \max \{$$
$$t_{pcq} + t_{mem} + t_{setup},$$
$$2(t_{RFread} + t_{mux} + t_{eq} + t_{AND} + t_{mux} + t_{setup}),$$
$$t_{pcq} + t_{mux} + t_{mux} + t_{ALU} + t_{setup},$$
$$t_{pcq} + t_{memwrite} + t_{setup},$$
$$2(t_{pcq} + t_{mux} + t_{RFwrite}) \}$$

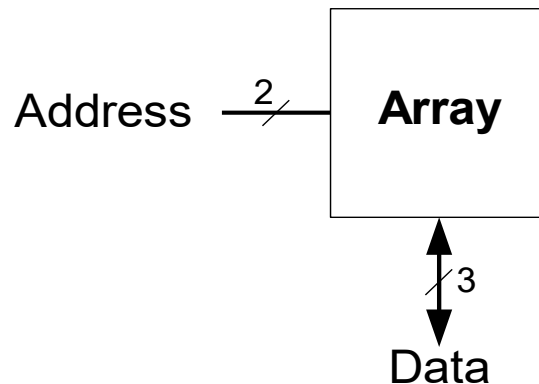
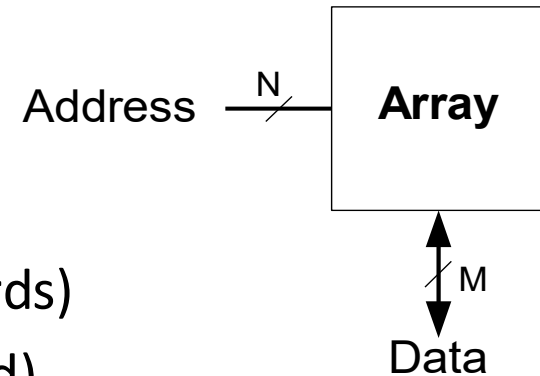
Memory Arrays

- Efficiently store large amounts of data
- 3 common types:
 - Dynamic random access memory (DRAM)
 - Static random access memory (SRAM)
 - Read only memory (ROM)
- M -bit data value read/ written at each unique N -bit address



Memory Arrays

- 2-dimensional array of bit cells
- Each bit cell stores one bit
- N address bits and M data bits:
 - 2^N rows and M columns
 - **Depth:** number of rows (number of words)
 - **Width:** number of columns (size of word)
 - **Array size:** depth $\times$ width = $2^N \times M$

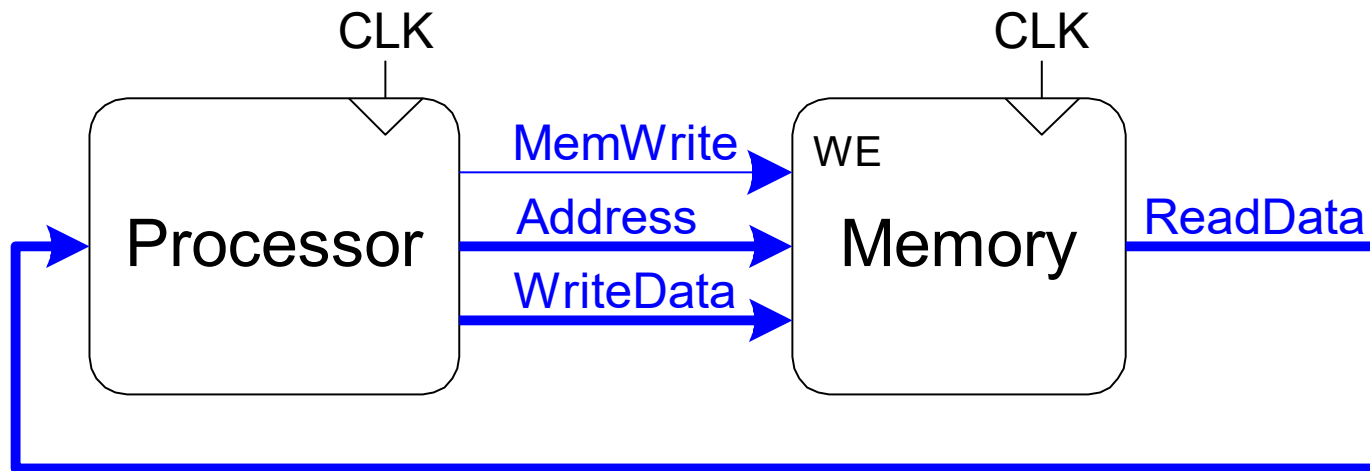


Address	Data			
11	0	1	0	depth ↑ ↓
10	1	0	0	
01	1	1	0	
00	0	1	1	
	width ←→			

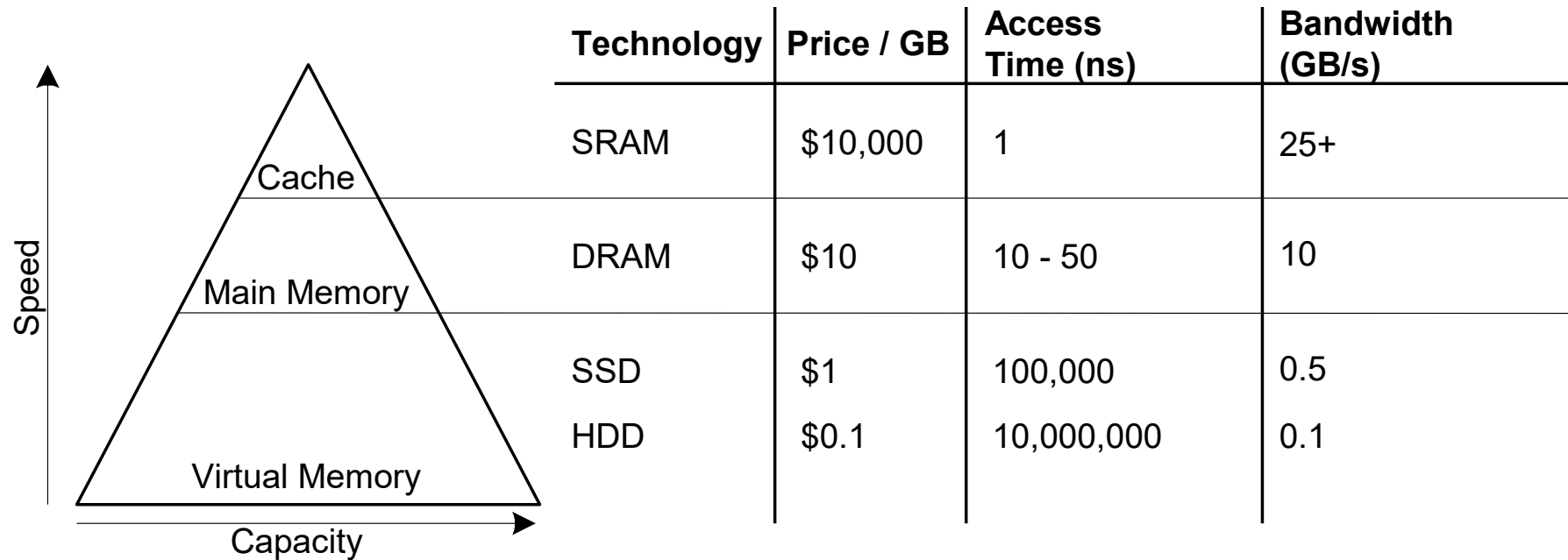
Memory

- Computer performance depends on:
 - Processor performance
 - Memory system performance

Memory Interface



Memory Hierarchy



Locality

Exploit locality to make memory accesses fast

- **Temporal Locality:**

- Locality in time
- If data used recently, likely to use it again soon
- **How to exploit:** keep recently accessed data in higher levels of memory hierarchy

- **Spatial Locality:**

- Locality in space
- If data used recently, likely to use nearby data soon
- **How to exploit:** when access data, bring nearby data into higher levels of memory hierarchy too

Memory Performance

- **Hit:** data found in that level of memory hierarchy
- **Miss:** data not found (must go to next level)

Hit Rate $= \# \text{ hits} / \# \text{ memory accesses}$
 $= 1 - \text{Miss Rate}$

Miss Rate $= \# \text{ misses} / \# \text{ memory accesses}$
 $= 1 - \text{Hit Rate}$

- **Average memory access time (AMAT):** average time for processor to access data

$$\text{AMAT} = t_{L1} + MR_{L1}[t_{L2} + MR_{L2}(t_{L3} + \dots)]$$